

**Laboratorio técnico CI/CD: Integración continua con GitHub Actions y entrega continua
con Jenkins**

Victor Enrique Cantor Beltrán

Universidad de La Sabana

Fundamentos de DevOps

María Fernanda Ochoa

18 de agosto de 2026

Laboratorio técnico CI/CD: Integración continua con GitHub Actions y entrega continua con Jenkins

El laboratorio implementa un flujo de integración continua y entrega continua para una aplicación web en Python. El repositorio GitHub se utiliza como fuente de verdad para el código, las pruebas, el Dockerfile, el workflow de GitHub Actions, el Jenkinsfile y los manifiestos de Kubernetes. La solución separa los controles de calidad temprana de las actividades de empaquetado y publicación, de manera coherente con la práctica de automatizar la entrega y mantener el pipeline bajo control de versiones.

Objetivos y alcance

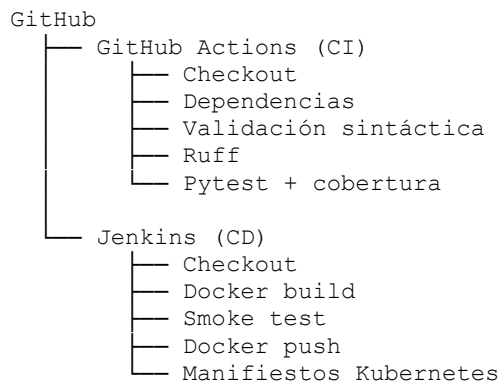
El objetivo general fue configurar dos pipelines complementarios que permitieran validar automáticamente el código de una aplicación web y preparar un artefacto contenerizado trazable para su entrega. GitHub Actions se empleó para la integración continua y Jenkins para la entrega continua.

- Ejecutar el pipeline de CI automáticamente ante eventos de push y pull request.
- Realizar checkout, instalación de dependencias, validación sintáctica, análisis estático y pruebas automatizadas.
- Aplicar un umbral de cobertura mínimo del 85 % como criterio de calidad.
- Construir una imagen Docker desde Jenkins y ejecutar un smoke test sobre el endpoint /health.
- Publicar la imagen en Docker Hub con credenciales administradas por Jenkins y tags trazables.
- Generar manifiestos Kubernetes con la imagen exacta producida por el pipeline.

Arquitectura de la solución

La arquitectura integra GitHub, GitHub Actions, Jenkins, Docker Hub y Kubernetes.

GitHub Actions permite definir workflows automatizados en YAML y asociarlos a eventos del repositorio (GitHub, Inc., s. f.). Jenkins Pipeline permite representar una cadena de entrega como código dentro de un Jenkinsfile versionado junto con la aplicación (Jenkins, s. f.). Docker se utiliza para empaquetar la aplicación mediante instrucciones declaradas en un Dockerfile (Docker, Inc., s. f.). Finalmente, el pipeline prepara manifiestos para un despliegue posterior sobre Kubernetes, plataforma que gestiona cargas contenerizadas de forma declarativa (Kubernetes, 2026).



Aplicación y estructura del repositorio

La aplicación está desarrollada en Python con Flask e incorpora un endpoint de salud que permite verificar su disponibilidad durante el pipeline. El repositorio mantiene separados los archivos de automatización, pruebas, documentación y manifiestos, facilitando la mantenibilidad y la trazabilidad.

```

actividad3-devops-cicd/
├── .github/workflows/ci.yml
├── Dockerfile
├── Jenkinsfile
├── README.md
├── app.py
├── pyproject.toml
├── requirements.txt
├── requirements-dev.txt
├── tests/
├── k8s/

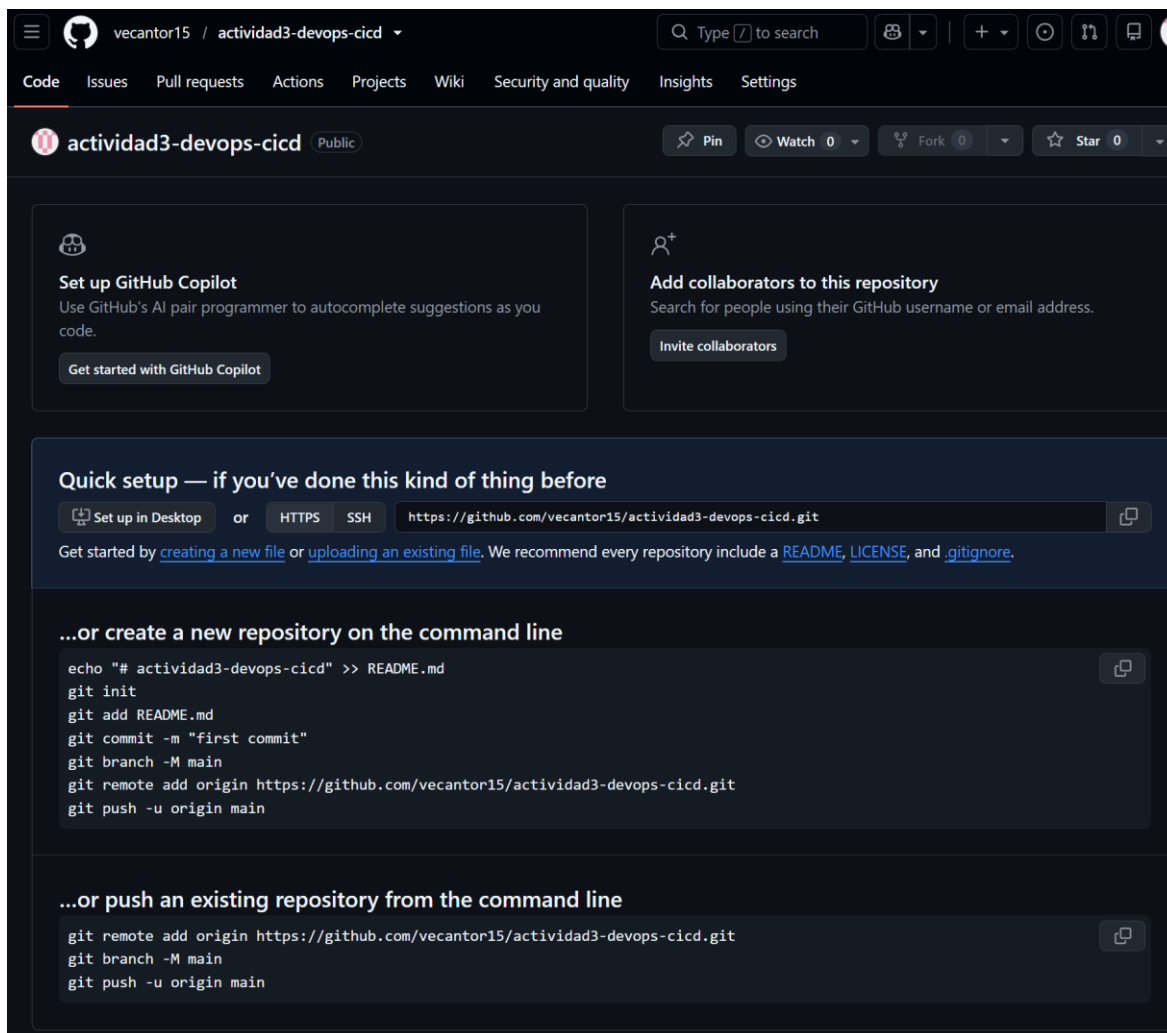
```

```

├── deployment.yaml
├── service.yaml
├── build/
└── docs/

```

Figura 1



Nota. Captura de pantalla del repositorio utilizado para alojar el código y los archivos de automatización del laboratorio.

Integración continua con GitHub Actions

El archivo `.github/workflows/ci.yml` define el workflow CI - Build, Test and Validate. El workflow se ejecuta ante push, pull_request y de forma manual mediante workflow_dispatch. GitHub establece que los workflows se definen en archivos YAML dentro del directorio `.github/workflows` y se componen de uno o más jobs y steps.

Tabla 1*Etapas implementadas en el pipeline de integración continua*

Etapas	Control	Propósito
1	Checkout del código	Obtiene la revisión exacta a validar.
2	Configurar Python 3.13	Estabiliza el runtime y habilita caché de pip.
3	Instalar dependencias	Reproduce el entorno de prueba.
4	Validación sintáctica	Detecta errores tempranos con compileall.
5	Análisis estático con Ruff	Comprueba calidad estática del código.
6	Pytest y cobertura	Ejecuta pruebas y exige cobertura mínima del 85 %.
7	Publicación de cobertura	Conserva coverage.xml como artefacto.

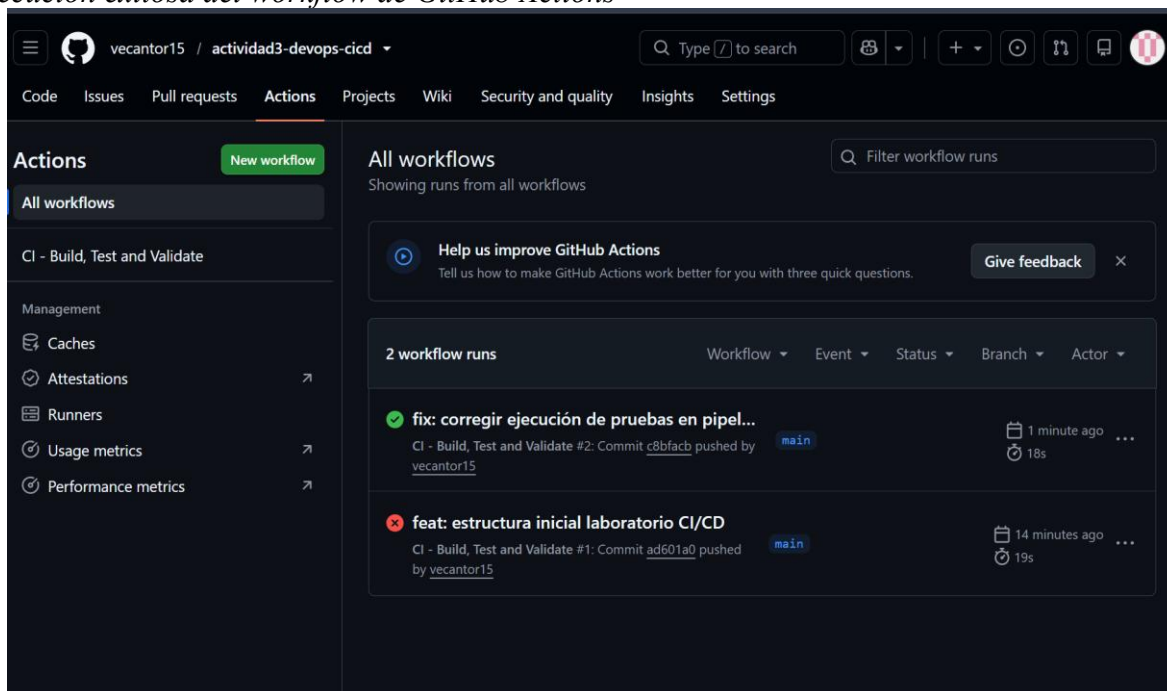
Nota. El contenido de la tabla sintetiza la configuración implementada en el repositorio del laboratorio.

Quality gate de pruebas

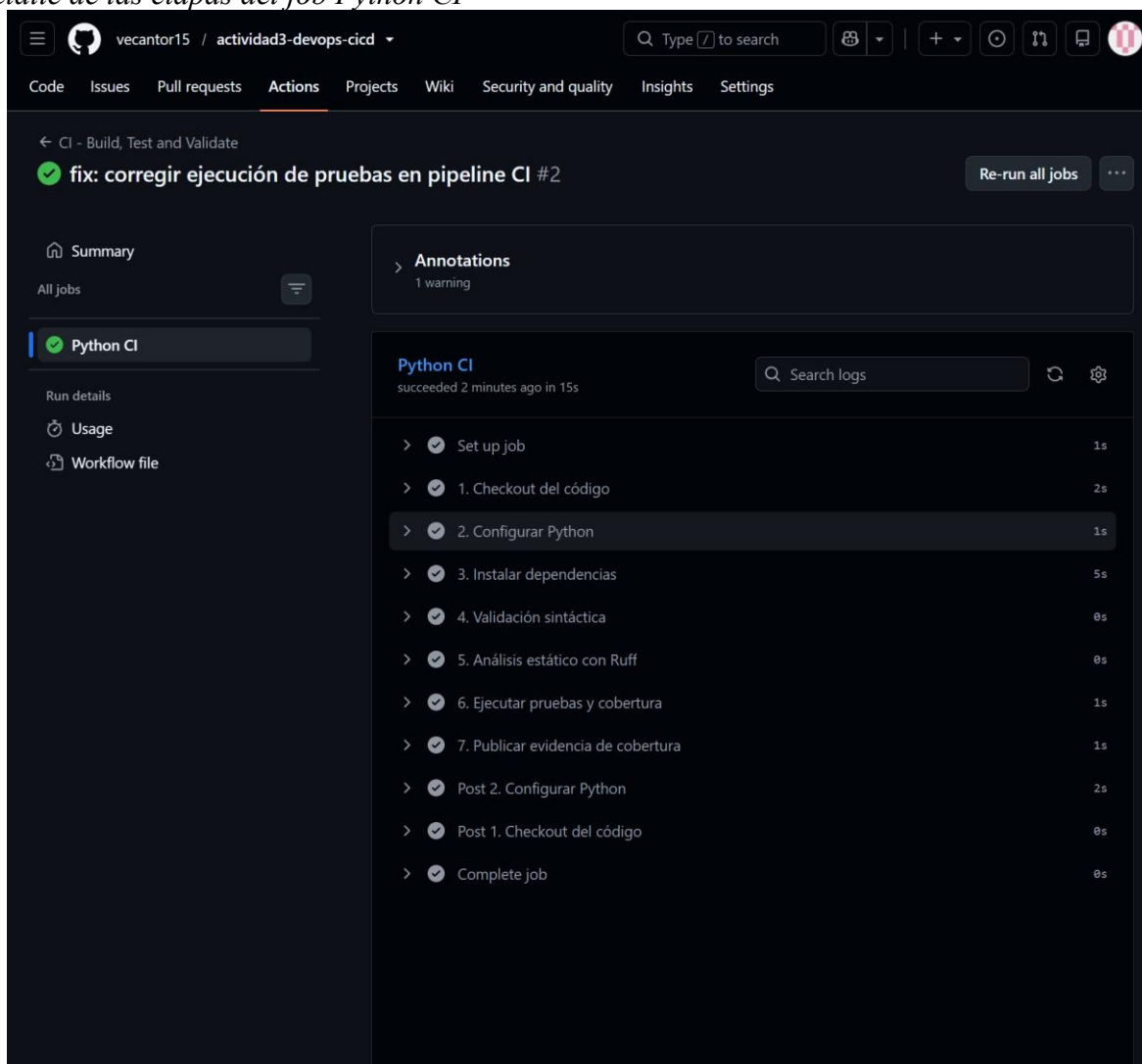
El pipeline utiliza Pytest con pytest-cov y establece un umbral mínimo de cobertura del 85 %. Esta decisión impide que una revisión con cobertura insuficiente avance como ejecución válida y materializa una práctica de retroalimentación temprana.

```
python -m pytest \
  --cov=app \
  --cov-report=term-missing \
  --cov-report=xml:coverage.xml \
  --cov-fail-under=85
```

Figura 2
Ejecución exitosa del workflow de GitHub Actions



Nota. La ejecución en verde confirma que el workflow CI fue procesado correctamente después de las correcciones realizadas.

Figura 3*Detalle de las etapas del job Python CI*

Nota. Se observan completadas las etapas de checkout, configuración de Python, instalación de dependencias, validación sintáctica, Ruff, pruebas con cobertura y publicación del artefacto.

Entrega continua con Jenkins

El pipeline de entrega continua se define en un Jenkinsfile declarativo. Jenkins describe Pipeline como un conjunto de capacidades para modelar procesos de entrega como código y recomienda versionar el Jenkinsfile en el sistema de control de fuentes (Jenkins, s. f.). En el laboratorio, el pipeline recibe parámetros para el namespace de Docker Hub, el repositorio de imágenes y la activación opcional del despliegue en Kubernetes.

Figura 4*Parámetros configurados para la ejecución en Jenkins***Pipeline actividad3-devops-cicd**

Esta ejecución requiere parámetros adicionales:

DOCKERHUB_NAMESPACE

Usuario u organización de Docker Hub

victorc901202

DOCKERHUB_REPOSITORY

Repositorio de imágenes en Docker Hub

devops-web-lab

☐ DEPLOY_TO_K8S

Fase opcional para el siguiente laboratorio: desplegar en Kubernetes

▶ Ejecución

Cancel

Nota. Los parámetros utilizados fueron DOCKERHUB_NAMESPACE = victorc901202, DOCKERHUB_REPOSITORY = devops-web-lab y DEPLOY_TO_K8S desactivado para esta fase del laboratorio.

Tabla 2*Stages definidos en el pipeline de entrega continua*

Stage	Responsabilidad
1. Checkout	Obtiene el repositorio y calcula el SHA corto del commit.
2. Validate Definition	Verifica Dockerfile, workflow CI y manifiestos Kubernetes.
3. Build Docker Image	Construye la imagen Docker de la aplicación.
4. Smoke Test Container	Levanta temporalmente la imagen y consulta /health.
5. Publish to Docker Hub	Autentica y publica el tag trazable y latest.
6. Prepare Kubernetes Manifest	Genera y archiva los manifiestos con la imagen publicada.
7. Deploy to Kubernetes	Stage opcional condicionado por DEPLOY_TO_K8S.

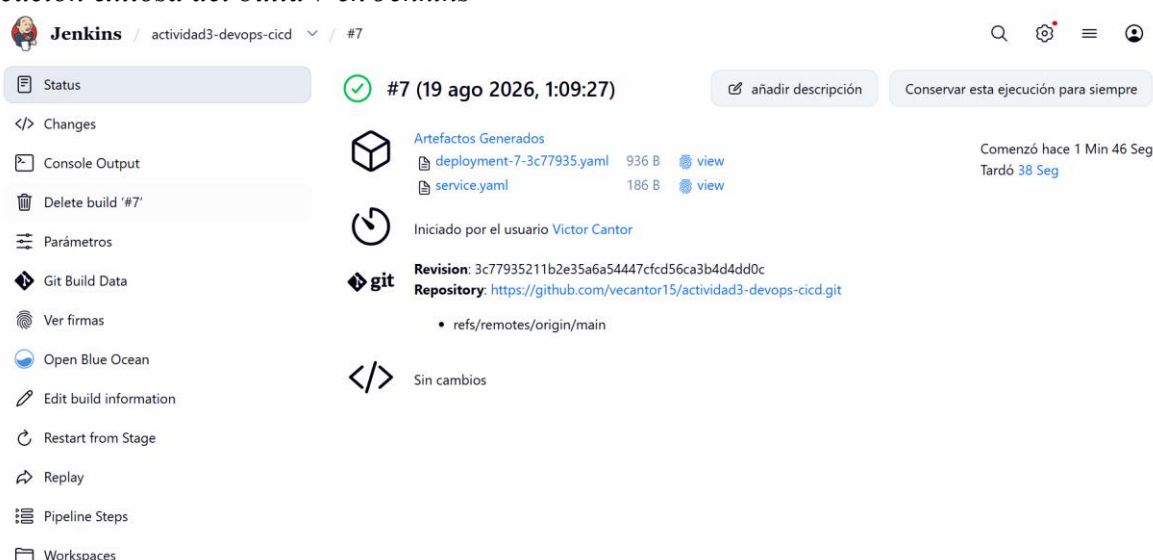
Nota. El contenido de la tabla sintetiza la configuración implementada en el repositorio del laboratorio.

Trazabilidad del artefacto

El tag de la imagen combina el número de build con los primeros siete caracteres del SHA del commit. La ejecución exitosa produjo el tag 7-3c77935, lo que permite relacionar el artefacto de Docker Hub con el build 7 de Jenkins y con el commit 3c77935 del repositorio. Esta estrategia facilita auditoría, diagnóstico y reproducibilidad.

Figura 5

Ejecución exitosa del build 7 en Jenkins



Nota. El build finalizó exitosamente, utilizó la revisión 3c77935211b2e35a6a54447cfd56ca3b4d4dd0c y archivó los manifiestos deployment-7-3c77935.yaml y service.yaml.

Contenerización y publicación en Docker Hub

El Dockerfile utiliza python:3.13-slim como imagen base, instala las dependencias de ejecución, crea un usuario no privilegiado y ejecuta la aplicación con Gunicorn. Además, define HEALTHCHECK contra el endpoint /health. Docker documenta Dockerfile como el archivo que contiene las instrucciones necesarias para construir una imagen y contempla instrucciones como FROM, RUN, USER, EXPOSE, HEALTHCHECK y CMD (Docker, Inc., s. f.).

```
FROM python:3.13-slim
...
RUN useradd --create-home --uid 10001 appuser
USER appuser
EXPOSE 8000
HEALTHCHECK ... /health
CMD ["gunicorn", "--bind", "0.0.0.0:8000", "--workers", "2", "app:app"]
```

La autenticación contra Docker Hub se realizó mediante un Personal Access Token almacenado en Jenkins con el identificador dockerhub-credentials. El secreto no forma parte del repositorio. Una vez superado el smoke test, Jenkins publica un tag trazable y actualiza latest para la rama principal.

Figura 6
Imágenes publicadas en Docker Hub

Repositories / devops-web-lab / Tags

victorc901202/devops-web-lab

Last pushed 8 minutes ago · Repository size: 43.4 MB · ☆0 · ↓25

Add a description

Add a category

Using 0 of 1 private repositories.

Docker commands

To push a new tag to this repository:

```
docker push victorc901202/devops-web-lab:tagname
```

General Tags Image Management Collaborators Webhooks Settings

Sort by Newest Filter tags Delete

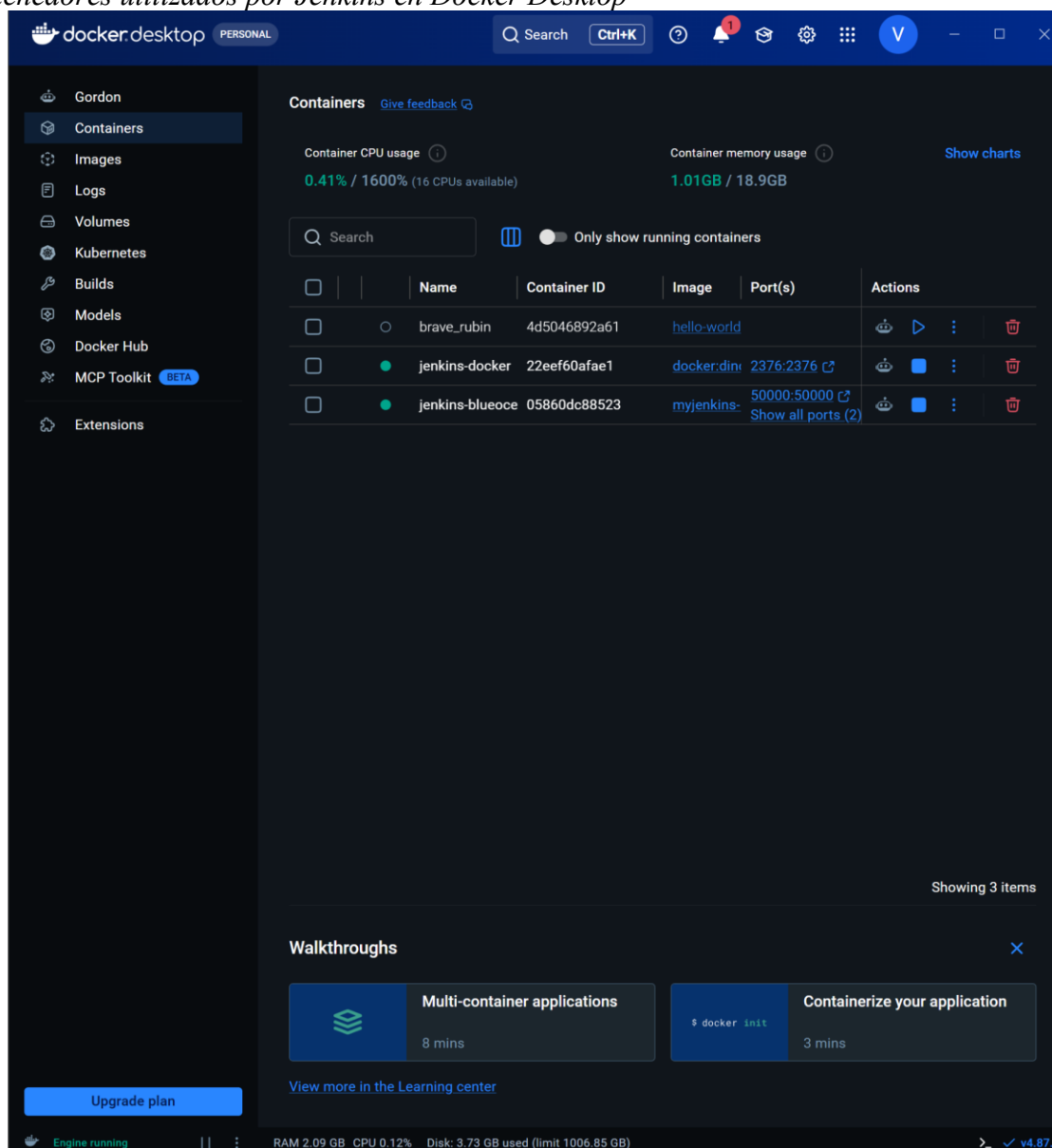
TAG	Digest	OS/ARCH	Last pull	Compressed size
latest	d3e9f8c80496	linux/amd64	less than 1 day	43.36 MB
7-3c77935	d3e9f8c80496	linux/amd64	less than 1 day	43.36 MB

Nota. El repositorio victorc901202/devops-web-lab contiene los tags latest y 7-3c77935. Ambos muestran el mismo digest, lo cual evidencia que corresponden a la misma imagen generada por el pipeline.

Infraestructura local de Jenkins y Docker

Jenkins fue ejecutado de forma contenerizada sobre Docker Desktop mediante una arquitectura Docker-in-Docker. El contenedor jenkins-blueocean alojó Jenkins y el contenedor jenkins-docker proporcionó el daemon Docker utilizado para construir, validar y publicar imágenes.

Figura 7
Contenedores utilizados por Jenkins en Docker Desktop



Nota. Se observan activos los contenedores jenkins-docker y jenkins-blueocean, utilizados para la ejecución local del pipeline de entrega continua.

Preparación para Kubernetes

El alcance del laboratorio deja preparada la fase de despliegue sin activarla. Jenkins sustituye el marcador IMAGE_PLACEHOLDER por el tag exacto de la imagen generada y archiva los archivos YAML como artefactos. Kubernetes permite describir un estado deseado mediante recursos declarativos y utiliza controladores para aproximar el estado real al estado especificado (Kubernetes, 2026).

```
sed "s#IMAGE_PLACEHOLDER#{REMOTE_IMAGE}:${IMAGE_TAG}#g" \
    k8s/deployment.yaml > build/deployment-${IMAGE_TAG}.yaml
cp k8s/service.yaml build/service.yaml
```

Seguridad y buenas prácticas

La implementación incorporó controles orientados a reducir exposición de secretos, errores de calidad y problemas de trazabilidad. El enfoque utilizado es coherente con la idea de integrar seguridad, confiabilidad y retroalimentación dentro del flujo de entrega, en lugar de tratarlas únicamente como actividades posteriores (Kim et al., 2021).

Tabla 3

Controles de seguridad y calidad implementados

Práctica	Implementación
Gestión de secretos	Docker Hub se autentica mediante Jenkins Credentials y PAT; el token no se versiona.
Mínimo privilegio en CI	GitHub Actions utiliza permissions: contents: read.
Contenedor no root	El Dockerfile ejecuta la aplicación con el usuario appuser.
Trazabilidad	Los tags combinan BUILD_NUMBER y SHA corto.
Validación previa a publicación	El smoke test consulta /health y detiene el pipeline ante falla.
Quality gate	Ruff, compileall, Pytest y cobertura mínima del 85 %.
Concurrencia	Jenkins deshabilita builds concurrentes y conserva un historial limitado.

Nota. El contenido de la tabla sintetiza la configuración implementada en el repositorio del laboratorio.

Incidentes técnicos y resolución

Durante la implementación se presentaron fallos que fueron tratados a partir de la evidencia de consola y de pruebas incrementales. Los incidentes principales y su resolución se resumen en la Tabla 4.

Tabla 4

Incidentes técnicos identificados y acciones correctivas

Incidente	Efecto	Resolución
Ejecución de Pytest	El workflow falló al invocar las pruebas.	Se utilizó <code>python -m pytest</code> conservando los parámetros de cobertura.
Opción <code>timestamps()</code> en Jenkins	Jenkins rechazó la opción declarativa.	Se retiró <code>timestamps()</code> y se mantuvieron <code>timeout</code> , <code>buildDiscarder</code> y <code>disableConcurrentBuilds</code> .
Resolución DNS en Docker-in-Docker	El build no resolvía <code>registry-1.docker.io</code> .	Se validó <code>resolv.conf</code> y la resolución mediante DNS internos y externos; posteriormente el build completó.
Autenticación de Docker Hub	La publicación requería una credencial válida.	Se creó un PAT con permisos de lectura/escritura y se almacenó en Jenkins.

Nota. El contenido de la tabla sintetiza la configuración implementada en el repositorio del laboratorio.

Correspondencia con los criterios de evaluación

Las evidencias obtenidas permiten demostrar que la solución no se limita a archivos de configuración estáticos. Se ejecutaron los pipelines, se identificaron errores, se aplicaron correcciones y se obtuvo una publicación real en Docker Hub.

Tabla 5

Matriz de cumplimiento del laboratorio

Criterio	Evidencia de cumplimiento
Pipeline CI funcional y automático	GitHub Actions ejecuta checkout, dependencias, <code>sintaxis</code> , <code>Ruff</code> , pruebas, cobertura y artefacto.
Stages de CD correctamente definidos	Jenkinsfile con checkout, validación, build, smoke test, push, preparación de manifiestos y deploy opcional.
Herramientas actuales y relevantes	GitHub Actions, Jenkins, Docker, Docker Hub, Pytest, <code>Ruff</code> y Kubernetes.
Documentación técnica	Documento técnico, repositorio, código versionado y capturas de ejecución.

Repositorio organizado y funcional	Separación de workflows, tests, k8s, docs y archivos de raíz.
------------------------------------	---

Nota. El contenido de la tabla sintetiza la configuración implementada en el repositorio del laboratorio.

Conclusiones

La solución implementada permitió separar claramente la integración continua de la entrega continua. GitHub Actions proporcionó la retroalimentación temprana sobre sintaxis, análisis estático, pruebas y cobertura, mientras Jenkins concentró la construcción de la imagen, el smoke test, la publicación y la preparación de los manifiestos.

La ejecución exitosa del build 7 demostró que el pipeline de entrega fue funcional y no solamente declarativo. La publicación de los tags latest y 7-3c77935 en Docker Hub estableció una relación verificable entre el código fuente, la ejecución de Jenkins y el artefacto entregable.

Finalmente, el laboratorio evidenció que la automatización CI/CD también exige diagnosticar problemas de entorno, compatibilidad de plugins, resolución de nombres y manejo de credenciales. La corrección iterativa de estos incidentes fortaleció el valor de la observabilidad y la retroalimentación continua dentro del proceso DevOps.

Referencias

Docker, Inc. (s. f.). *Dockerfile reference*. Recuperado el 18 de agosto de 2026, de

<https://docs.docker.com/reference/dockerfile/>

GitHub, Inc. (s. f.). *Workflow syntax for GitHub Actions*. GitHub Docs. Recuperado el 18 de

agosto de 2026, de <https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax>

Jenkins. (s. f.). *Pipeline*. Recuperado el 18 de agosto de 2026, de

<https://www.jenkins.io/doc/book/pipeline/>

Kim, G., Humble, J., Debois, P., & Willis, J. (2021). *The DevOps handbook: How to create world-class agility, reliability, & security in technology organizations* (2nd ed.). IT Revolution Press.

Kubernetes. (2026, 18 de junio). *Deployments*.

<https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>